

Recommender Systems Using Matrix Factorization

Almir Mustafic

Faculty of Electrical Engineering
University of Sarajevo
Sarajevo, Bosnia and Herzegovina

Merim Jusufbegovic

Faculty of Electrical Engineering
University of Sarajevo
Sarajevo, Bosnia and Herzegovina

Abstract—English. Explicit rating prediction remains a core task in movie recommendation. This paper evaluates biased matrix factorization (BMF) trained with Alternating Least Squares (ALS) on MovieLens Latest Small. The model combines latent user and item vectors with global, user, and item bias terms. On a per-user hold-out split, it achieves test Root Mean Squared Error (RMSE) 0.89 and Mean Absolute Error (MAE) 0.67, compared with 1.05 and 0.83 for a global-mean baseline. The analysis covers training behaviour, sample held-out predictions, and an illustrative top-10 recommendation list. The results confirm that bias-augmented latent factor models estimate missing ratings on sparse explicit feedback more accurately than a mean predictor.

Bosanski. Predviđanje eksplicitne ocjene ostaje ključni problem u procesu preporučivanja filmova. Ovaj rad analizira faktORIZACIJU matrica sa odstupanjima (BMF) treniranu metodom naizmjeničnih najmanjih kvadrata (ALS) na datasetu MovieLens Latest Small. Model kombinuje latentne vektore korisnika i stavki (filmova) sa globalnim, korisničkim i odstupanjima stavki. Na podjeli dataseta po korisniku, postiže testnu srednju kvadratnu grešku (RMSE) od 0,89 i srednju apsolutnu grešku (MAE) od 0,67, u poređenju sa 1,05 i 0,83 za osnovni model globalne srednje vrijednosti. Analiza obuhvata trening modela, predviđanja na osnovu testnog uzorka i ilustrativnu listu 10 najboljih preporuka. Rezultati pokazuju da modeli latentnih faktora prošireni odstupanjima preciznije procjenjuju nedostajuće ocjene na rijetkim eksplicitnim povratnim informacijama nego model koji predviđa srednju vrijednost.

Index Terms—recommender systems, matrix factorization, collaborative filtering, biased matrix factorization, alternating least squares, MovieLens

I. INTRODUCTION

A. Recommender Systems

Streaming services, online shops, and similar platforms list far more titles and products than anyone could browse in full. Recommender systems address that problem by suggesting items from past interactions [4], for instance through “You might also like” panels. The same idea appears in e-commerce and online learning, where the goal is to surface relevant items without asking the user to search the full catalog [1], [5]. Embedding-based collaborative filtering has been the dominant approach in this setting for many years [6]. Accurate rating prediction on sparse feedback remains important because many applications rank unseen items from predicted scores rather than from raw interaction counts alone [3], [4].

B. Collaborative Filtering

Recommender systems are often grouped into collaborative filtering and content-based filtering [4]. Content-based methods rely on item attributes such as genre or description [4].

Classic collaborative filtering recommends from user-item interaction patterns rather than item descriptions [4]. If two users agreed on many movies, the system can suggest titles that one of them has not seen yet [4]. Matrix factorization is a latent-factor form of collaborative filtering [2], [3]. Neighborhood methods compare users or items directly and can work well on dense local patterns [3]. Latent factor models compress the full rating history into short vectors and scale better when the matrix is very sparse, which is the case for MovieLens [1], [2].

C. Matrix Factorization and Latent User-Item Factors

Ratings are stored in a sparse matrix $\mathbf{R}$. Each known value r_{ui} is the score user u gave item i [3]. Matrix factorization (MF) assigns a short latent vector to every user and every item [3], [6]. Their dot product estimates user-item compatibility [3], [6]. The latent dimension F is much smaller than the user count M or item count N [3], which makes the model practical on large sparse datasets [1], [2]. Netflix Prize work helped establish matrix factorization as a standard tool for explicit rating prediction at scale [4].

BMF adds a global mean μ , a user bias b_u , and an item bias b_i to that dot product [2], [3]. MovieLens-style data often contains generous raters, strict raters, and movies that are generally liked or disliked [2]. Bias terms capture those baseline rating levels [2], [3]. ALS with those bias terms gives strong results on MovieLens [1], and the BiasedMF model is a standard explicit-feedback baseline in comparative studies [2]. Section 2 describes BMF with ALS, and Section 3 evaluates it on MovieLens Latest Small against a global-mean baseline.

D. Missing Rating Prediction

Most cells in $\mathbf{R}$ are empty because users rate only a small part of the catalog [3]. The learning task is to guess scores for unrated pairs and rank the highest ones as recommendations [3], [4]. Feedback can be explicit, such as 1–5 stars, or implicit, such as click logs [3], [4]. Explicit feedback is easier to interpret but sparse. Implicit feedback is abundant but ambiguous because a click does not reveal how much the user liked an item [4]. Comparative work reports that matrix factorization fits explicit star ratings better than several graph-based alternatives [10]. Cold-start users or items with few ratings remain difficult even for strong factorization models [10]. The evaluation in Section 3 uses

explicit MovieLens ratings only and focuses on the warm-user setting, where each test user also has train history.

E. Selected Literature

Explicit rating prediction on sparse user-item matrices remains a standard collaborative-filtering task. Most pairs are unobserved, yet systems must estimate missing scores and rank unseen items from those estimates [3], [4]. Low-rank matrix factorization is the usual approach because it compresses interaction history into latent user and item vectors [3], [6]. On MovieLens-style data, a plain dot-product model is often not enough. Some users rate high, others low, and some titles are generally liked or disliked regardless of user taste [2]. Biased matrix factorization adds a global mean plus user and item offsets so baseline rating level is separated from latent compatibility [2], [3]. The BiasedMF model is treated as a core explicit-feedback baseline in comparative MovieLens studies [1], [2], [5].

Training and evaluation follow the same line of work. ALS fits BMF by solving regularized least-squares subproblems and is widely used on MovieLens-scale explicit data [1], [2]. RMSE and MAE remain common hold-out metrics in explicit rating-prediction studies, with RMSE especially associated with the Netflix Prize tradition [4], [5]. Neural and graph-based recommenders can improve some ranking settings [6], [10], but bias-augmented linear factorization is still a strong, interpretable reference for explicit star ratings on sparse data [1], [2], [6]. Other extensions, such as reliability weighting, rating-centrality weighting, weighted latent factors, dynamic priors, and distributed ALS [7]–[9], [11], [12], build on the same matrix-factorization idea but are outside the scope of this study. Sections 2 and 3 apply BMF with ALS on MovieLens Latest Small under a fixed per-user split, decompose the rating matrix into latent factors, and predict and rank missing movie ratings in a practical recommendation example.

II. USED ALGORITHM

We chose BMF with ALS to predict missing star ratings and rank unseen movies on MovieLens Latest Small. This section describes our Python implementation of that model and training procedure, and Section 3 reports the test errors and recommendation output from running it on the dataset.

A. User-Item Rating Matrix

Collaborative filtering for explicit ratings starts from a user-item matrix $\mathbf{R}$ [3], [4]. Each row corresponds to one user, each column to one item, and each stored entry is a rating r_{ui} . Unrated pairs are missing values to be estimated [3], [4]. On MovieLens Latest Small only about 1.7% of all user-item pairs contain a rating. Real collaborative filtering (CF) datasets are sparse in this way because each user rates only a small fraction of the catalog [2], [3].

B. Matrix Factorization and Biased Matrix Factorization

Plain MF predicts compatibility through latent vectors alone,

$$\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i, \quad (1)$$

where $\mathbf{p}_u$ and $\mathbf{q}_i$ are the latent vectors of user u and item i [3], [6].

BMF adds baseline terms [2], [4], [5],

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i. \quad (2)$$

Here μ is the mean train rating, b_u captures how user u usually rates relative to μ , and b_i captures how item i is usually rated relative to μ [2], [3]. The dot product covers the specific user-item match once those offsets are removed [3], [6].

MovieLens star ratings show clear user-level and item-level shifts, so BMF is preferred over plain MF [1], [2]. ALS is used because each subproblem reduces to a regularized least-squares update with a closed-form solution [1]. That makes training stable on explicit rating data and easy to compare with published BiasedMF formulations [1], [2]. Stochastic gradient alternatives can also fit matrix factorization models, but ALS is widely used in comparative MovieLens studies [1], [2].

C. Latent Vector Initialization

Each latent vector has length k . In the evaluation, $k = 10$. Before ALS starts, μ is set to the mean train rating, biases are zero, and latent vectors are drawn from a normal distribution with scale 0.01. Zero initialization is avoided because it keeps all dot products at zero at the start [1], [3].

D. Alternating Least Squares Training

Training follows ALS [1], [2]. One iteration updates all user vectors, then all item vectors, then all bias terms [1]. For user u , with rated items $N(u)$, the update uses residuals

$$e_{ui} = r_{ui} - \mu - b_u - b_i \quad (3)$$

and solves

$$\left(\sum_{i \in N(u)} \mathbf{q}_i \mathbf{q}_i^\top + \lambda \mathbf{I} \right) \mathbf{p}_u = \sum_{i \in N(u)} e_{ui} \mathbf{q}_i. \quad (4)$$

Item vectors are updated with the same pattern after swapping the roles of users and items. Each regularized normal equation is solved directly. Training runs for 40 ALS iterations on the train split only. During implementation, ALS iteration counts from 10 to 70 were tested in steps of 10. Because test RMSE changed only minimally beyond 40, 40 iterations were used for the reported run.

Each ALS iteration applies the initialization in the previous subsection, then performs a user pass, an item pass, and a bias pass. In the user pass, every user vector is updated from train residuals and the regularized normal equations above. The item pass repeats the same pattern with users and items swapped. In the bias pass, each b_u and b_i is set to the mean residual after subtracting μ and the latent dot product, and μ is updated from the remaining mean error. After these three passes, the regularized squared-error objective $\mathcal{L}$ is recorded on the train split to monitor convergence. The bias update follows the standard Biased Matrix Factorization (BMF) approach [1], [2].

E. Regularization and Training Objective

In this implementation, only the latent factor matrices receive an L_2 penalty. Biases are not regularized, as in typical BiasedMF setups [1], [2]. The regularization strength is $\lambda = 10$. ALS minimizes

$$\mathcal{L} = \sum_{(u,i) \in \text{train}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|\mathbf{P}\|_F^2 + \|\mathbf{Q}\|_F^2). \quad (5)$$

The value of $\mathcal{L}$ after each iteration is tracked during training. On MovieLens Latest Small, this objective falls from about 51,200 to about 42,200 on the train split. It measures fit on observed ratings, not held-out test error.

F. Recommendation Generation

Held-out rating prediction is evaluated in Section 3. Recommendation applies the same formula to train-unseen items. For user u , items already rated in train are excluded, remaining items are scored, and the top N predictions form the recommendation list [2]–[4]. The results section uses user index 0 and $N = 10$ as an illustrative case.

G. Practical Application

In practice, a movie recommender can learn from historical ratings and then use the trained model to rank unseen titles for each user [4]. Training estimates latent user and item vectors and the bias terms that adjust for general rating tendencies. Inference scores unseen titles for the active user and returns the highest predictions. RMSE and MAE on held-out ratings measure prediction quality, while top- N ranking shows how missing scores translate into recommendations. Content features from movie metadata were not used here, but they could help new movies with no ratings yet [4], [10].

III. EXPERIMENTAL EVALUATION

A. Dataset and Experimental Setup

MovieLens Latest Small is a standard benchmark for explicit rating prediction [13]. It contains 610 users, 9,724 movies, and 100,836 star ratings from 1 to 5. The dataset was collected from the MovieLens recommender service and has been used widely to compare collaborative filtering methods because the ratings are explicit, publicly available, and large enough to expose sparsity effects while remaining easy to reproduce [13]. Only about 1.7% of possible user-item pairs are observed, which matches the sparse regime described in matrix factorization literature [3], [4]. Explicit feedback in the 1–5 range is standard in this line of work [3], [4]. Table I lists the fixed model and split settings used in all results below.

B. Data Preparation and Train-Test Split

A per-user train-test split is applied. Each user’s ratings are shuffled with random seed 42 and 20% are assigned to test. Users with one rating remain in train only, so every test user also appears in train. This design avoids empty user vectors at test time. The split contains 80,896 train ratings and 19,940 test ratings.

The settings in Table I follow standard matrix factorization practice on MovieLens-scale data [1], [2]. Latent rank, ALS training, and L_2 regularization on factor matrices are frequent design choices in literature [1], [2], [6]. Forty iterations were suitable for the training objective to flatten on this dataset, as shown in Fig. 1 on the next page.

Table I
EXPERIMENTAL SETTINGS

Setting	Value	Description
Latent rank k	10	Dimension of $\mathbf{p}_u$ and $\mathbf{q}_i$
Regularization λ	10	L_2 penalty on factor matrices
ALS iterations	40	Training rounds
Test fraction	0.20	Per-user hold-out
Random seed	42	Split reproducibility
Top- N recommendations	10	Illustrative ranking length

C. Evaluation Metrics

Rating prediction quality is measured with RMSE and MAE on the held-out test set. Both metrics are widely used for explicit rating prediction [2], [4]. RMSE penalizes large errors more heavily and was central to the Netflix Prize evaluation framework [4]. MAE gives a direct interpretation as average absolute star error [2].

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (r_i - \hat{r}_i)^2}, \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |r_i - \hat{r}_i|. \quad (6)$$

The baseline predictor returns the train global mean μ for every test rating [1], [2], [4]. Table II on the next page shows that BMF beats this baseline on both metrics, which indicates that latent factors capture user-item structure beyond the dataset-wide average.

D. Test Set Results

Table II summarizes the dataset and test performance. BMF with ALS improves on the global-mean baseline for both metrics. RMSE drops from 1.05 to 0.89 and MAE from 0.83 to 0.67 on the 1–5 star scale. Both metrics fall by 0.16 in absolute terms, which is about 15.2% relative improvement for RMSE and 19.3% for MAE against the baseline. For a simple BMF model against a global-mean predictor, that is a clear but moderate gain rather than a large improvement in absolute prediction quality. Fig. 1 shows the error comparison, training objective, experimental settings, and eight sample test predictions with movie titles.

The training objective decreases steadily across ALS iterations. Most of the drop occurs in the first ten rounds, after which the curve flattens. This suggests that 40 iterations are sufficient for convergence on this dataset.

E. Example Predictions

Table III on the next page lists eight held-out test cases. Most predictions stay within a few tenths of a star. *East of Eden* and *Groundhog Day* are representative close matches.

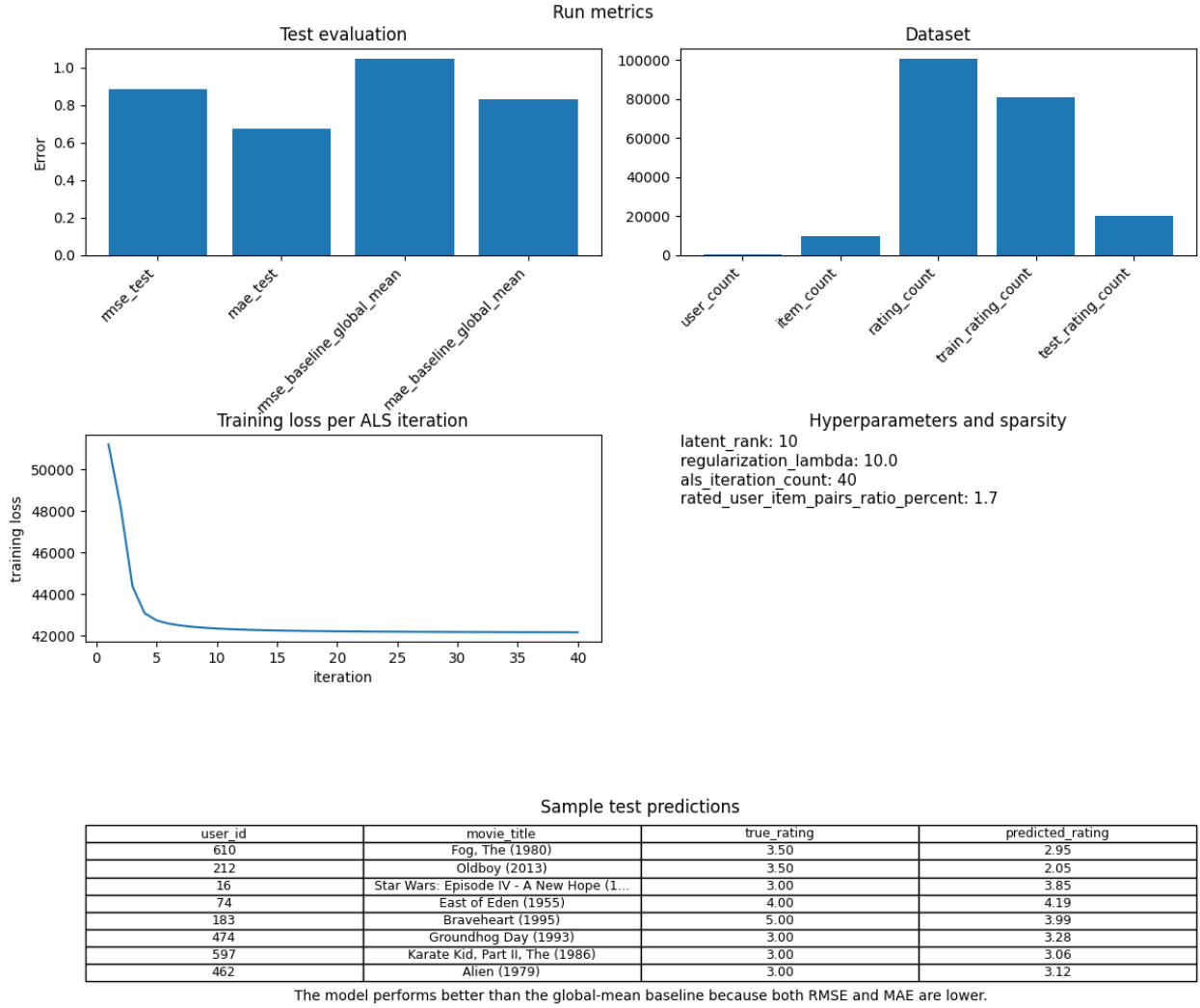


Figure 1. Experimental results on MovieLens Latest Small. Top row, dataset counts and test RMSE/MAE with baseline comparison. Bottom row, training objective per ALS iteration and experimental settings. The table lists eight random test rows with movie titles, true ratings, and model predictions.

Table II
RUN SUMMARY ON MOVIELENS LATEST SMALL

Quantity	Value
Users	610
Items	9,724
Ratings	100,836
Matrix density	1.7%
Train ratings	80,896
Test ratings	19,940
Test RMSE	0.89
Test MAE	0.67
Baseline RMSE	1.05
Baseline MAE	0.83

is a larger error, with true rating 3.5 and prediction 2.05. *Braveheart* shows the opposite pattern, where a 5.0 rating is predicted closer to 4.0. These cases illustrate that BMF captures general user-item compatibility well on average but still misses unusual or highly individual preferences on some pairs [2].

F. Comparison with Reported Benchmarks

The absolute test numbers depend on split rule, seed, and hyperparameters, so they should be compared only qualitatively to published MovieLens research [2], [5]. Table IV places the reported run against this selected literature. Prior work on explicit MovieLens splits ranks BiasedMF among the stronger linear models [2], while simple regularized matrix factorization remains a standard baseline on the same dataset

Karate Kid, Part II and *Alien* are almost exact. *Oldboy (2013)*

Table III
SAMPLE TEST PREDICTIONS

user_id	movie_title	true	pred.
610	Fog, The (1980)	3.5	2.95
212	Oldboy (2013)	3.5	2.05
16	Star Wars: Episode IV - A New Hope (1977)	3.0	3.85
74	East of Eden (1955)	4.0	4.19
183	Braveheart (1995)	5.0	3.99
474	Groundhog Day (1993)	3.0	3.28
597	Karate Kid, Part II, The (1986)	3.0	3.06
462	Alien (1979)	3.0	3.12

family [5]. Against that background, ALS with bias terms is a commonly used approach that reaches competitive accuracy on MovieLens-like explicit feedback [1]. In this run, BMF with ALS is better than the global-mean baseline and it reached sub-one-star MAE on held-out ratings (Table II). The fixed per-user split differs from some published setups, but the relative improvement matches prior reports for bias-augmented factorization on sparse star-rating data [1], [2], [10].

Table IV
COMPARISON WITH THEMES IN THE SELECTED LITERATURE

Source theme	Finding in this evaluation
BiasedMF on MovieLens [2]	Bias terms improve over mean predictor (Table II)
ALS with biases [1]	Stable ALS training; RMSE 0.89 vs baseline 1.05 (Table II)
Explicit vs implicit [10]	Star ratings fit MF better than graph baselines
RMSE/MAE protocol [4], [5]	Both metrics reported on hold-out data
Top- N from predicted scores [3], [4]	Unseen items ranked by $\hat{r}_{ui}$

G. Top-10 Recommendations

Table V lists the top 10 train-unseen movies for user index 0, ranked by the model’s raw prediction scores. The scores are ranking outputs, not calibrated 1–5 star estimates: the model was not constrained to the MovieLens rating range, so values above 5.0 are expected and should be read as relative preference, not literal star ratings [3], [4]. For top- N recommendation, only item order matters. Titles come from the MovieLens metadata join used elsewhere in Section 3.

H. Discussion

The quantitative results support three points that appear repeatedly in the matrix factorization literature [1]–[4]. First, explicit MovieLens data contain strong user-level and item-level baseline effects, and bias terms help separate those effects from latent compatibility [2], [3]. Second, low-rank latent vectors beat the global-mean baseline on held-out stars in this run (Table II), consistent with prior MovieLens reports [1], [2]. Third, high predicted scores on unseen items provide a direct ranking basis for top- N recommendation [3], [4].

The remaining errors in Table III and the item cold-start issue noted above show the limits of pure collaborative

Table V
TOP-10 TRAIN-UNSEEN RECOMMENDATIONS FOR USER INDEX 0, RANKED BY THE MODEL’S RAW PREDICTION SCORES

Rank	Movie title	Score
1	Strawberry and Chocolate (1993)	7.37
2	Swimming with Sharks (1995)	7.37
3	Inkheart (2008)	7.26
4	Hitman: Agent 47 (2015)	7.18
5	xXx: State of the Union (2005)	6.88
6	Demon Knight (1995)	6.87
7	Trainwreck (2015)	6.68
8	San Andreas (2015)	6.54
9	Over the Top (1987)	6.50
10	He’s Just Not That Into You (2009)	6.48

filtering. Some user-item pairs are poorly represented in the sparse train matrix, and items with no training ratings cannot receive meaningful latent estimates [10]. These limitations are consistent with prior comparative studies and motivate hybrid or content-aware extensions rather than a change in the core BMF formulation [4], [10].

I. Limitations

The per-user split ensures train history for every test user, but about 750 test items still had no train ratings. Those items keep weakly estimated latent vectors and unreliable predictions, which is a known item cold-start limitation of collaborative filtering [10]. All hyperparameters in Table I ($k = 10$, $\lambda = 10$, 40 ALS iterations, seed 42) were set in advance and held fixed for the entire study. No cross-validation or grid search was used to tune them on MovieLens Latest Small. The top-10 list in Table V is ranked by the model’s raw prediction scores. Several values exceed 5.0, so they must not be read as literal MovieLens star ratings. For that example, only item order is meaningful. The evaluation covers offline rating prediction and top- N ranking and does not address online serving, model refresh, or hybrid content features.

Future work should tune hyperparameters with cross-validation, clip predicted scores to the 1–5 range, and join movie titles to internal item indexes in recommendation exports for readability. Incorporating movie metadata could also reduce item cold-start error [4], [10].

IV. CONCLUSION

A. Summary of Results

This paper reviewed matrix factorization approaches for explicit rating prediction and evaluated biased matrix factorization with ALS on MovieLens Latest Small. On held-out test data, the model outperformed a global-mean baseline (Table II), and the training objective decreased steadily over 40 ALS iterations. Sample predictions and a top-10 recommendation list show how latent factors support both rating estimation and item ranking on sparse explicit feedback.

The results align with the literature on bias-augmented matrix factorization [1], [2]. User and item biases capture baseline rating effects that plain dot-product models must learn

implicitly. Regularization remains important on sparse train data. A per-user split avoids empty user vectors but still leaves item cold-start cases.

B. Possible Improvements

Cross-validation over k , λ , and the number of ALS iterations would likely reduce test error further. Item-side metadata or a split rule that guarantees train exposure for every item would address cold-start movies. Clipping predictions to the 1–5 range would make recommendation scores easier to interpret in production settings.

REFERENCES

- [1] “Latent Geometry of Taste: Scalable Low-Rank Matrix Factorization for Recommender Systems,” arXiv:2601.03466, 2026.
- [2] “Comprehensive Evaluation of Matrix Factorization Models for Collaborative Filtering Recommender Systems,” arXiv:2410.17644, 2024.
- [3] “An Introduction to Matrix Factorization and Factorization Machines in Recommendation System, and Beyond,” arXiv:2203.11026, 2022.
- [4] “An Introduction to Collaborative Filtering through the Lens of the Netflix Prize,” *Knowledge and Information Systems*, 2025.
- [5] “UserReg: A Simple but Strong Model for Rating Prediction,” ICASSP / arXiv:2102.07601, 2021.
- [6] “Neural Collaborative Filtering vs. Matrix Factorization Revisited,” RecSys / arXiv:2005.09683, 2020.
- [7] “Providing Reliability in Recommender Systems through Bernoulli Matrix Factorization,” *Information Sciences* / arXiv:2006.03481, 2021.
- [8] “Optimization Matrix Factorization Recommendation Algorithm Based on Rating Centrality,” arXiv:1806.07678, 2018.
- [9] “Weighted-SVD: Matrix Factorization with Weights on the Latent Factors,” arXiv:1710.00482, 2017.
- [10] “A Comparative Study of Matrix Factorization and Random Walk with Restart in Recommender Systems,” IEEE BigData / arXiv:1708.09088, 2017.
- [11] “Dynamic Matrix Factorization with Priors on Unknown Values,” KDD / arXiv:1507.06452, 2015.
- [12] “Distributed Matrix Factorization using Asynchronous Communication,” arXiv:1705.10633, 2017.
- [13] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems*, 2015.